

# ISYE 6740 Fall 2026 — Homework 2

Neil Shukla

|   |                                                         |    |
|---|---------------------------------------------------------|----|
| 1 | Concept Questions (10 points)                           | 2  |
| 2 | Graph Laplacian and GNN (20 points)                     | 4  |
| 3 | Correlated features (motivation for PCA) (20 points)    | 8  |
| 4 | PCA: Food consumption in European countries (10 points) | 12 |
| 5 | Order of faces using ISOMAP (20 points)                 | 14 |
| 6 | Eigenfaces and simple face recognition (20 points)      | 20 |
| 7 | To subtract or not to subtract (Bonus: 10 points)       | 23 |

## 1. Concept Questions (10 points)

### 1.1 The first principal component is the top eigenvector of the sample covariance (5 points)

**Setup.** Let  $\mathbf{x}_1, \dots, \mathbf{x}_m \in \mathbb{R}^n$  be the data,  $\boldsymbol{\mu} = \frac{1}{m} \sum_{i=1}^m \mathbf{x}_i$  the sample mean, and

$$C = \frac{1}{m} \sum_{i=1}^m (\mathbf{x}_i - \boldsymbol{\mu})(\mathbf{x}_i - \boldsymbol{\mu})^T \in \mathbb{R}^{n \times n}$$

the sample covariance matrix. We must show that

$$\mathbf{v} = \arg \max_{\mathbf{w}: \|\mathbf{w}\| \leq 1} \frac{1}{m} \sum_{i=1}^m (\mathbf{w}^T \mathbf{x}_i - \mathbf{w}^T \boldsymbol{\mu})^2$$

is the eigenvector of  $C$  associated with its largest eigenvalue  $\lambda_1$ .

**Step 1: Rewrite the objective as a quadratic form in  $C$ .** Since  $\mathbf{w}^T \mathbf{x}_i - \mathbf{w}^T \boldsymbol{\mu} = \mathbf{w}^T (\mathbf{x}_i - \boldsymbol{\mu})$  is a scalar, it equals its own transpose, so

$$(\mathbf{w}^T (\mathbf{x}_i - \boldsymbol{\mu}))^2 = \mathbf{w}^T (\mathbf{x}_i - \boldsymbol{\mu}) (\mathbf{x}_i - \boldsymbol{\mu})^T \mathbf{w}.$$

Summing and pulling  $\mathbf{w}$  outside the (linear) sum,

$$f(\mathbf{w}) := \frac{1}{m} \sum_{i=1}^m (\mathbf{w}^T \mathbf{x}_i - \mathbf{w}^T \boldsymbol{\mu})^2 = \mathbf{w}^T \left[ \frac{1}{m} \sum_{i=1}^m (\mathbf{x}_i - \boldsymbol{\mu})(\mathbf{x}_i - \boldsymbol{\mu})^T \right] \mathbf{w} = \mathbf{w}^T C \mathbf{w}.$$

So the problem is  $\max_{\|\mathbf{w}\| \leq 1} \mathbf{w}^T C \mathbf{w}$ : we seek the unit direction along which the projected data has maximum sample variance.

**Step 2: Properties of  $C$ .**  $C$  is symmetric, since each  $(\mathbf{x}_i - \boldsymbol{\mu})(\mathbf{x}_i - \boldsymbol{\mu})^T$  is symmetric. It is positive semidefinite because  $\mathbf{w}^T C \mathbf{w} = \frac{1}{m} \sum_i (\mathbf{w}^T (\mathbf{x}_i - \boldsymbol{\mu}))^2 \geq 0$  for all  $\mathbf{w}$ . By the spectral theorem,  $C = U \Lambda U^T$  with  $U = [\mathbf{u}_1, \dots, \mathbf{u}_n]$  orthogonal and  $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_n)$ , where  $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_n \geq 0$ .

**Step 3: The constraint is active.**  $f$  is homogeneous of degree 2:  $f(c\mathbf{w}) = c^2 f(\mathbf{w})$ . If  $0 < \|\mathbf{w}\| < 1$ , then  $\mathbf{w}/\|\mathbf{w}\|$  is feasible and  $f(\mathbf{w}/\|\mathbf{w}\|) = f(\mathbf{w})/\|\mathbf{w}\|^2 \geq f(\mathbf{w})$ . So a maximizer can always be taken on the boundary  $\|\mathbf{w}\| = 1$ , and the problem is equivalent to  $\max_{\mathbf{w}^T \mathbf{w} = 1} \mathbf{w}^T C \mathbf{w}$ .

**Step 4: Lagrangian / first-order conditions.** Form the Lagrangian  $\mathcal{L}(\mathbf{w}, \lambda) = \mathbf{w}^T C \mathbf{w} - \lambda(\mathbf{w}^T \mathbf{w} - 1)$ . Setting the gradient to zero (using  $C = C^T$ ):

$$\nabla_{\mathbf{w}} \mathcal{L} = 2C\mathbf{w} - 2\lambda\mathbf{w} = \mathbf{0} \implies C\mathbf{w} = \lambda\mathbf{w}.$$

Every stationary point is therefore a unit eigenvector of  $C$ , with the multiplier  $\lambda$  equal to its eigenvalue.

**Step 5: Evaluate the objective at stationary points.** If  $C\mathbf{w} = \lambda\mathbf{w}$  and  $\mathbf{w}^T \mathbf{w} = 1$ , then

$$f(\mathbf{w}) = \mathbf{w}^T C \mathbf{w} = \mathbf{w}^T (\lambda \mathbf{w}) = \lambda \mathbf{w}^T \mathbf{w} = \lambda.$$

The variance captured by an eigenvector equals its eigenvalue. The largest achievable value among stationary points is  $\lambda_1$ , attained at  $\mathbf{w} = \mathbf{u}_1$ .

**Step 6: Global optimality (not just stationarity).** For any unit  $\mathbf{w}$ , let  $\mathbf{z} = U^T \mathbf{w}$ . Then  $\|\mathbf{z}\| = \|\mathbf{w}\| = 1$  since  $U$  is orthogonal, and

$$\mathbf{w}^T C \mathbf{w} = \mathbf{w}^T U \Lambda U^T \mathbf{w} = \mathbf{z}^T \Lambda \mathbf{z} = \sum_{j=1}^n \lambda_j z_j^2 \leq \lambda_1 \sum_{j=1}^n z_j^2 = \lambda_1.$$

Equality holds when  $\mathbf{z} = \mathbf{e}_1$ , i.e.  $\mathbf{w} = U \mathbf{e}_1 = \mathbf{u}_1$ . So  $\lambda_1$  is the global maximum of the objective over the feasible set and it is attained by  $\mathbf{u}_1$ .

**Conclusion.**  $\mathbf{v} = \arg \max_{\|\mathbf{w}\| \leq 1} \frac{1}{m} \sum_i (\mathbf{w}^T \mathbf{x}_i - \mathbf{w}^T \boldsymbol{\mu})^2 = \mathbf{u}_1$ , the eigenvector of the sample covariance  $C$  associated with its *largest* eigenvalue  $\lambda_1$ . The maximal projected variance equals  $\lambda_1$ . The eigenvector is unique up to sign when  $\lambda_1 > \lambda_2$ .

## 1.2 Non-uniqueness of the eigendecomposition of $A = \begin{pmatrix} 3 & 0 \\ 0 & 2 \end{pmatrix}$ (5 points)

**Eigenvalues.** The characteristic polynomial is

$$\det(A - \lambda I) = \det \begin{pmatrix} 3 - \lambda & 0 \\ 0 & 2 - \lambda \end{pmatrix} = (3 - \lambda)(2 - \lambda) = 0 \implies \lambda \in \{3, 2\}.$$

**Eigenvectors.** For  $\lambda = 3$ :  $(A - 3I)\mathbf{u} = \begin{pmatrix} 0 & 0 \\ 0 & -1 \end{pmatrix} \begin{pmatrix} u_1 \\ u_2 \end{pmatrix} = \mathbf{0} \Rightarrow u_2 = 0$ . The eigenspace is  $\{c(1, 0)^T : c \neq 0\}$ .

For  $\lambda = 2$ :  $(A - 2I)\mathbf{u} = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix} \begin{pmatrix} u_1 \\ u_2 \end{pmatrix} = \mathbf{0} \Rightarrow u_1 = 0$ . The eigenspace is  $\{c(0, 1)^T : c \neq 0\}$ .

Because each eigenvector is determined only up to a nonzero scalar, and the ordering of the eigenpairs is arbitrary, there is freedom in writing  $A = U \Lambda U^{-1}$ . Two distinct decompositions follow.

**Decomposition 1 (standard basis, descending order).**

$$U_1 = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \quad \Lambda_1 = \begin{pmatrix} 3 & 0 \\ 0 & 2 \end{pmatrix}, \quad U_1^{-1} = I.$$

*Verification:*  $U_1 \Lambda_1 U_1^{-1} = I \Lambda_1 I = \begin{pmatrix} 3 & 0 \\ 0 & 2 \end{pmatrix} = A$ . ✓

**Decomposition 2 (sign-flipped first eigenvector, reversed order).** Take the eigenvectors  $\mathbf{u} = (0, 1)^T$  for  $\lambda = 2$  and  $\mathbf{u} = (-1, 0)^T$  for  $\lambda = 3$  (a valid eigenvector since it is  $c = -1$  times  $(1, 0)^T$ ):

$$U_2 = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}, \quad \Lambda_2 = \begin{pmatrix} 2 & 0 \\ 0 & 3 \end{pmatrix}.$$

$U_2$  is orthogonal ( $U_2^T U_2 = I$ , columns of unit length and orthogonal), so  $U_2^{-1} = U_2^T = \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix}$ .

*Verification:*

$$U_2 \Lambda_2 = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 2 & 0 \\ 0 & 3 \end{pmatrix} = \begin{pmatrix} 0 & -3 \\ 2 & 0 \end{pmatrix}, \quad U_2 \Lambda_2 U_2^T = \begin{pmatrix} 0 & -3 \\ 2 & 0 \end{pmatrix} \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix} = \begin{pmatrix} 3 & 0 \\ 0 & 2 \end{pmatrix} = A. \quad \checkmark$$

Equivalently, column by column:  $A(0, 1)^T = (0, 2)^T = 2(0, 1)^T$  and  $A(-1, 0)^T = (-3, 0)^T = 3(-1, 0)^T$ , so  $A U_2 = U_2 \Lambda_2$ .

**Why these are distinct.**  $U_1 \neq U_2$  and  $\Lambda_1 \neq \Lambda_2$ , yet both products reproduce  $A$  exactly.

**Conclusion.**  $A = U_1 \Lambda_1 U_1^{-1} = U_2 \Lambda_2 U_2^{-1}$  with  $(U_1, \Lambda_1) \neq (U_2, \Lambda_2)$ , so the eigendecomposition is *not unique*. The non-uniqueness has two sources.

- (i) Scaling: if  $A\mathbf{u} = \lambda\mathbf{u}$  then  $A(c\mathbf{u}) = \lambda(c\mathbf{u})$  for any  $c \neq 0$ , and  $c = -1$  gives a sign flip even under unit-norm normalization.
- (ii) Ordering: permuting the columns of  $U$  together with the diagonal of  $\Lambda$  leaves  $U\Lambda U^{-1}$  unchanged.

The eigenvalues, as a set  $\{3, 2\}$ , and the eigenspaces are unique. The particular matrices  $U, \Lambda$  are not.

## 2. Graph Laplacian and GNN (20 points)

**Notation.**  $G = (V, E)$  is an undirected graph with  $n$  nodes, adjacency  $A$  ( $A_{ij} = 1$  if  $i \sim j$ , else 0, and  $A_{ii} = 0$ ), degrees  $d_i = \sum_j A_{ij}$ , degree matrix  $D = \text{diag}(d_1, \dots, d_n)$ , and Laplacian  $L = D - A$ . Entry-wise,

$$L_{ij} = \begin{cases} d_i, & i = j, \\ -1, & i \neq j, i \sim j, \\ 0, & \text{otherwise.} \end{cases}$$

In particular,  $L_{ij} \neq 0$  only if  $i = j$  or  $i$  and  $j$  are 1-hop neighbors.

### 2(1) Neighborhood aggregation (10 points)

**(a) Show that**  $(L\mathbf{x})_i = d_i x_i - \sum_{j \sim i} x_j$ .

By the definition of matrix-vector multiplication and  $L = D - A$ ,

$$(L\mathbf{x})_i = \sum_{j=1}^n L_{ij} x_j = \sum_{j=1}^n D_{ij} x_j - \sum_{j=1}^n A_{ij} x_j.$$

Since  $D$  is diagonal,  $\sum_j D_{ij} x_j = D_{ii} x_i = d_i x_i$ . Since  $A_{ij} = 1$  exactly when  $j \sim i$  and 0 otherwise,  $\sum_j A_{ij} x_j = \sum_{j \sim i} x_j$ . Hence

$$(L\mathbf{x})_i = d_i x_i - \sum_{j \sim i} x_j = \sum_{j \sim i} (x_i - x_j).$$

The last form uses  $d_i = |\{j : j \sim i\}|$ . The new value at node  $i$  depends *only* on  $x_i$  and the values at its 1-hop neighbors. Multiplying by  $L$  is therefore a local aggregation operator: it compares each node with its neighbors, like a discrete (negative) second derivative.

**(b) Why  $L^2\mathbf{x}$  reaches paths of length up to 2.**

*Entry-wise argument.* Write  $L^2\mathbf{x} = L(L\mathbf{x})$  and apply part (a) to the vector  $\mathbf{y} = L\mathbf{x}$ :

$$(L^2\mathbf{x})_i = d_i y_i - \sum_{j \sim i} y_j, \quad y_j = d_j x_j - \sum_{k \sim j} x_k.$$

Substituting,

$$(L^2 \mathbf{x})_i = \underbrace{d_i^2 x_i - d_i \sum_{j \sim i} x_j}_{\text{node } i \text{ and its 1-hop neighbors}} - \sum_{j \sim i} d_j x_j + \underbrace{\sum_{j \sim i} \sum_{k \sim j} x_k}_{\text{walks } i \rightarrow j \rightarrow k}.$$

The final double sum runs over every walk  $i \rightarrow j \rightarrow k$  of length 2, so it contains the values  $x_k$  of nodes  $k$  that are two hops from  $i$  (neighbors of neighbors). These nodes do not appear in  $(L \mathbf{x})_i$ . It also contains  $k = i$ , walks going out and back, which is why the self-term picks up extra weight. No node at distance  $\geq 3$  can appear, since every term is reached by at most two edges.

*Matrix-entry argument.*  $(L^2)_{ik} = \sum_j L_{ij} L_{jk}$ . A summand is nonzero only if  $L_{ij} \neq 0$  (so  $j = i$  or  $j \sim i$ ) and  $L_{jk} \neq 0$  (so  $k = j$  or  $k \sim j$ ). Therefore  $(L^2)_{ik} \neq 0$  requires a walk of length  $\leq 2$  from  $i$  to  $k$ . Equivalently,  $(L^2)_{ik} = 0$  whenever  $\text{dist}(i, k) > 2$ .

*Concrete check (3-node path 1 – 2 – 3).* Nodes 1 and 3 are not adjacent, so  $L_{13} = 0$ . But (see part 2)  $(L^2)_{13} = 1 \neq 0$ , because of the walk  $1 \rightarrow 2 \rightarrow 3$ . Explicitly,  $(L^2 \mathbf{x})_1 = 2x_1 - 3x_2 + x_3$ , so node 1 now receives information from node 3, two hops away.

### General statement (by induction).

**Lemma.** For every integer  $k \geq 0$ ,  $(L^k)_{ij} = 0$  whenever  $\text{dist}(i, j) > k$ .

*Proof.* For  $k = 0$ ,  $L^0 = I$  is nonzero only on the diagonal (distance 0). Assume the claim for  $k$ . Then  $(L^{k+1})_{ij} = \sum_\ell (L^k)_{i\ell} L_{\ell j}$ . A nonzero term needs  $\text{dist}(i, \ell) \leq k$  and  $(\ell = j \text{ or } \ell \sim j)$ , so  $\text{dist}(\ell, j) \leq 1$ . By the triangle inequality  $\text{dist}(i, j) \leq k + 1$ . Hence  $(L^{k+1})_{ij} = 0$  whenever  $\text{dist}(i, j) > k + 1$ .  $\square$

So  $L^k \mathbf{x}$  mixes information from walks of length up to  $k$ , i.e. the  $k$ -hop neighborhood of each node.

### (c) Why a polynomial in $L$ aggregates multiple hops in one shot.

A degree- $K$  polynomial filter is

$$p_\theta(L) \mathbf{x} = \sum_{k=0}^K \theta_k L^k \mathbf{x} \quad \text{or, in ChebNet,} \quad g_\theta(L) \mathbf{x} \approx \sum_{k=0}^K \theta_k T_k(\tilde{L}) \mathbf{x}, \quad \tilde{L} = \frac{2}{\lambda_{\max}} L - I.$$

- **Each term covers a fixed radius.** By the lemma,  $\theta_0 \mathbf{x}$  contributes node  $i$  itself,  $\theta_1 L \mathbf{x}$  contributes its 1-hop neighborhood,  $\theta_2 L^2 \mathbf{x}$  contributes its 2-hop neighborhood, and so on up to  $K$  hops.
- **The sum covers all radii at once.** Adding these terms gives, at node  $i$ , a weighted combination of all nodes within  $K$  hops, where  $\theta_k$  controls how much the  $k$ -hop information counts. A single layer therefore produces a multi-scale summary of the neighborhood, with no need to stack  $K$  separate 1-hop layers. The filter is also *exactly*  $K$ -localized:  $(p_\theta(L))_{ij} = 0$  for  $\text{dist}(i, j) > K$ .
- **The same holds for Chebyshev polynomials.**  $T_k$  is a polynomial of degree  $k$ .  $\tilde{L} = \frac{2}{\lambda_{\max}} L - I$  has the same sparsity pattern as  $L$  plus the diagonal, so  $T_k(\tilde{L})$  is a combination of  $I, L, \dots, L^k$  and has the same  $k$ -hop support. The rescaling maps the eigenvalues of  $L$  from  $[0, \lambda_{\max}]$  to  $[-1, 1]$ , where Chebyshev polynomials are bounded and numerically stable.
- **The operation is efficient and needs no eigenvectors.** The recurrence

$$T_k(\tilde{L}) \mathbf{x} = 2\tilde{L} T_{k-1}(\tilde{L}) \mathbf{x} - T_{k-2}(\tilde{L}) \mathbf{x},$$

with  $T_0(\tilde{L})\mathbf{x} = \mathbf{x}$  and  $T_1(\tilde{L})\mathbf{x} = \tilde{L}\mathbf{x}$ , computes all terms with  $K$  sparse matrix–vector products. The cost is  $O(K|E|)$ , versus  $O(n^3)$  for the eigendecomposition used in spectral clustering.

- **Link to spectral clustering.** With  $L = U\Lambda U^T$ ,  $p_\theta(L) = U p_\theta(\Lambda) U^T$ . The polynomial is a *spectral filter* that reweights the Laplacian eigenvectors, the same eigenvectors used for clustering, by  $p_\theta(\lambda)$ . Small- $\lambda$  eigenvectors vary smoothly over the graph, and large- $\lambda$  eigenvectors oscillate. Learning  $\theta$  learns which graph frequencies to keep, while evaluation happens entirely in the vertex domain.

**Conclusion.**  $L$  mixes 1-hop information,  $L^k$  mixes information along walks of length  $\leq k$ , and a learnable linear combination  $\sum_{k=0}^K \theta_k T_k(\tilde{L})$  therefore aggregates node features from the entire  $K$ -hop neighborhood in a single layer. The filter is  $K$ -localized, cheap to evaluate, and requires no eigendecomposition.

## 2(2) Small example calculation: the 3-node path (10 points)

The graph is the path  $1 - 2 - 3$  (edges  $\{1, 2\}$  and  $\{2, 3\}$ ).

**Adjacency matrix.**

$$A = \begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix}.$$

**Degree matrix.** Row sums of  $A$ :  $d_1 = 1$ ,  $d_2 = 2$ ,  $d_3 = 1$ .

$$D = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 1 \end{pmatrix}.$$

**Graph Laplacian.**

$$L = D - A = \begin{pmatrix} 1 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 1 \end{pmatrix}.$$

*Sanity checks:*  $L$  is symmetric and each row sums to 0, so  $L\mathbf{1} = \mathbf{0}$  and  $\lambda = 0$  is an eigenvalue. The eigenvalues of  $L$  are  $\{0, 1, 3\}$ , so  $\lambda_{\max} = 3$ .

$L^2$ . Computing  $(L^2)_{ij} = \sum_k L_{ik} L_{kj}$  entry by entry:

$$\begin{aligned} (L^2)_{11} &= (1)(1) + (-1)(-1) + (0)(0) = 2, & (L^2)_{12} &= (1)(-1) + (-1)(2) + (0)(-1) = -3, \\ (L^2)_{13} &= (1)(0) + (-1)(-1) + (0)(1) = 1, & (L^2)_{22} &= (-1)(-1) + (2)(2) + (-1)(-1) = 6, \\ (L^2)_{23} &= (-1)(0) + (2)(-1) + (-1)(1) = -3, & (L^2)_{33} &= (0)(0) + (-1)(-1) + (1)(1) = 2. \end{aligned}$$

The remaining entries follow from symmetry ( $L^2$  is symmetric because  $L$  is):  $(L^2)_{21} = -3$ ,  $(L^2)_{31} = 1$ ,  $(L^2)_{32} = -3$ .

$$L^2 = \begin{pmatrix} 2 & -3 & 1 \\ -3 & 6 & -3 \\ 1 & -3 & 2 \end{pmatrix}.$$

*Check:* rows sum to 0 ( $L^2 \mathbf{1} = L(L\mathbf{1}) = \mathbf{0}$ ), the matrix is symmetric, and the eigenvalues are  $\{0^2, 1^2, 3^2\} = \{0, 1, 9\}$ , with trace  $2 + 6 + 2 = 10 = 0 + 1 + 9$ . ✓

$$T_2(L) = 2L^2 - I.$$

$$T_2(L) = 2 \begin{pmatrix} 2 & -3 & 1 \\ -3 & 6 & -3 \\ 1 & -3 & 2 \end{pmatrix} - \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 3 & -6 & 2 \\ -6 & 11 & -6 \\ 2 & -6 & 3 \end{pmatrix}.$$

*Check:* the eigenvalues are  $2\lambda^2 - 1$  for  $\lambda \in \{0, 1, 3\}$ , i.e.  $\{-1, 1, 17\}$ , with sum  $17 = 3 + 11 + 3 = \text{tr } T_2(L)$ . ✓

### Interpretation of the result.

- $L_{13} = L_{31} = 0$  because nodes 1 and 3 are not adjacent, but  $(L^2)_{13} = (T_2(L))_{13} \neq 0$  because of the 2-hop walk  $1 \rightarrow 2 \rightarrow 3$ . This is exactly the 2-hop propagation from part 2(1)(b).
- For  $\mathbf{x} = (x_1, x_2, x_3)^T$ , the three ChebNet ( $K = 2$ ) building blocks are

$$T_0(L)\mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix}, \quad T_1(L)\mathbf{x} = L\mathbf{x} = \begin{pmatrix} x_1 - x_2 \\ -x_1 + 2x_2 - x_3 \\ -x_2 + x_3 \end{pmatrix}, \quad T_2(L)\mathbf{x} = \begin{pmatrix} 3x_1 - 6x_2 + 2x_3 \\ -6x_1 + 11x_2 - 6x_3 \\ 2x_1 - 6x_2 + 3x_3 \end{pmatrix}.$$

The output at node 1,  $\theta_0 x_1 + \theta_1(x_1 - x_2) + \theta_2(3x_1 - 6x_2 + 2x_3)$ , uses information from the whole 2-hop neighborhood.

**Remark (rescaled Laplacian actually used by ChebNet).** The question asks for  $T_2(L)$ , given above. For completeness, the filter in ChebNet applies  $T_k$  to  $\tilde{L} = \frac{2}{\lambda_{\max}}L - I$  with  $\lambda_{\max} = 3$ :

$$\tilde{L} = \frac{2}{3}L - I = \begin{pmatrix} -\frac{1}{3} & -\frac{2}{3} & 0 \\ -\frac{2}{3} & \frac{1}{3} & -\frac{2}{3} \\ 0 & -\frac{2}{3} & -\frac{1}{3} \end{pmatrix}, \quad \tilde{L}^2 = \begin{pmatrix} \frac{5}{9} & 0 & \frac{4}{9} \\ 0 & 1 & 0 \\ \frac{4}{9} & 0 & \frac{5}{9} \end{pmatrix}, \quad T_2(\tilde{L}) = 2\tilde{L}^2 - I = \begin{pmatrix} \frac{1}{9} & 0 & \frac{8}{9} \\ 0 & 1 & 0 \\ \frac{8}{9} & 0 & \frac{1}{9} \end{pmatrix}.$$

The eigenvalues of  $\tilde{L}$  are  $\{-1, -\frac{1}{3}, 1\} \subset [-1, 1]$ , as intended. The nonzero  $(1, 3)$  entry again shows 2-hop mixing.

### Summary of requested matrices.

$$A = \begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix}, \quad D = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 1 \end{pmatrix}, \quad L = \begin{pmatrix} 1 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 1 \end{pmatrix}, \quad L^2 = \begin{pmatrix} 2 & -3 & 1 \\ -3 & 6 & -3 \\ 1 & -3 & 2 \end{pmatrix}, \quad T_2(L) = \begin{pmatrix} 3 & -6 & 2 \\ -6 & 11 & -6 \\ 2 & -6 & 3 \end{pmatrix}.$$

### 3. Correlated features (motivation for PCA) (20 points)

**Setup.** For  $i = 1, \dots, m$  (with  $m = 10$ ),  $x_{1,i}$  is the centered number of bedrooms and  $x_{2,i}$  is the centered number of bathrooms, with the exact linear relation  $x_{1,i} = 1.5x_{2,i}$ . The design matrix and sample second-moment matrix are

$$X = \begin{pmatrix} \mathbf{1}^T \\ \mathbf{x}_1^T \\ \mathbf{x}_2^T \end{pmatrix} = \begin{pmatrix} 1 & 1 & \cdots & 1 \\ x_{1,1} & x_{1,2} & \cdots & x_{1,m} \\ x_{2,1} & x_{2,2} & \cdots & x_{2,m} \end{pmatrix} \in \mathbb{R}^{3 \times m}, \quad G = \frac{1}{m} X X^T \in \mathbb{R}^{3 \times 3},$$

and we write  $T = \sum_{i=1}^m x_{2,i}$  and  $S = \sum_{i=1}^m x_{2,i}^2$ . Throughout we assume the features are not identically zero, so  $S > 0$ .

#### 3.1 $G$ in terms of $T$ and $S$ , and $\text{rank}(G) = 2$ (10 points)

**Step 1: Entries of  $XX^T$ .** The  $(r, c)$  entry of  $XX^T$  is the inner product of rows  $r$  and  $c$  of  $X$ :

$$XX^T = \begin{pmatrix} \mathbf{1}^T \mathbf{1} & \mathbf{1}^T \mathbf{x}_1 & \mathbf{1}^T \mathbf{x}_2 \\ \mathbf{x}_1^T \mathbf{1} & \mathbf{x}_1^T \mathbf{x}_1 & \mathbf{x}_1^T \mathbf{x}_2 \\ \mathbf{x}_2^T \mathbf{1} & \mathbf{x}_2^T \mathbf{x}_1 & \mathbf{x}_2^T \mathbf{x}_2 \end{pmatrix} = \begin{pmatrix} m & \sum_i x_{1,i} & \sum_i x_{2,i} \\ \sum_i x_{1,i} & \sum_i x_{1,i}^2 & \sum_i x_{1,i} x_{2,i} \\ \sum_i x_{2,i} & \sum_i x_{1,i} x_{2,i} & \sum_i x_{2,i}^2 \end{pmatrix}.$$

**Step 2: Substitute  $x_{1,i} = 1.5x_{2,i}$ .**

$$\begin{aligned} \sum_i x_{1,i} &= 1.5 \sum_i x_{2,i} = 1.5 T, & \sum_i x_{1,i}^2 &= 2.25 \sum_i x_{2,i}^2 = 2.25 S, \\ \sum_i x_{1,i} x_{2,i} &= 1.5 \sum_i x_{2,i}^2 = 1.5 S, & \sum_i x_{2,i}^2 &= S. \end{aligned}$$

**Step 3: Assemble  $G$ .**

$$G = \frac{1}{m} \begin{pmatrix} m & 1.5 T & T \\ 1.5 T & 2.25 S & 1.5 S \\ T & 1.5 S & S \end{pmatrix} \stackrel{m=10}{=} \begin{pmatrix} 1 & 0.15 T & 0.1 T \\ 0.15 T & 0.225 S & 0.15 S \\ 0.1 T & 0.15 S & 0.1 S \end{pmatrix}$$

Because the features are centered,  $T = \sum_i x_{2,i} = m\bar{x}_2 = 0$  in this problem. The general form above holds for any  $T$ , and with  $T = 0$  it simplifies to

$$G|_{T=0} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 2.25 S/m & 1.5 S/m \\ 0 & 1.5 S/m & S/m \end{pmatrix}.$$

**Step 4:  $\text{rank}(G) \leq 2$ .** Denote the columns of  $mG$  by  $\mathbf{g}_1, \mathbf{g}_2, \mathbf{g}_3$ . Then

$$\mathbf{g}_2 = \begin{pmatrix} 1.5T \\ 2.25S \\ 1.5S \end{pmatrix} = 1.5 \begin{pmatrix} T \\ 1.5S \\ S \end{pmatrix} = 1.5 \mathbf{g}_3,$$

so the columns are linearly dependent. Equivalently, the nonzero vector  $\mathbf{n} = (0, 2, -3)^T$  lies in the null space:

$$G\mathbf{n} = \frac{1}{m} \begin{pmatrix} 2(1.5T) - 3T \\ 2(2.25S) - 3(1.5S) \\ 2(1.5S) - 3S \end{pmatrix} = \mathbf{0}.$$

Hence  $\det G = 0$  and  $\text{rank}(G) \leq 2$ . The root cause is that the row  $\mathbf{x}_1^T = 1.5 \mathbf{x}_2^T$  of  $X$  is redundant, so  $\text{rank}(X) \leq 2$  and  $\text{rank}(XX^T) = \text{rank}(X) \leq 2$ .

**Step 5:**  $\text{rank}(G) \geq 2$ . It suffices to exhibit a nonsingular  $2 \times 2$  submatrix. Take rows and columns  $\{1, 3\}$  of  $mG$ :

$$\det \begin{pmatrix} m & T \\ T & S \end{pmatrix} = mS - T^2 = m \sum_{i=1}^m (x_{2,i} - \bar{x}_2)^2.$$

This is strictly positive unless all  $x_{2,i}$  are equal. By Cauchy–Schwarz,  $T^2 = (\mathbf{1}^T \mathbf{x}_2)^2 \leq m \|\mathbf{x}_2\|^2 = mS$ , with equality only when  $\mathbf{x}_2 \propto \mathbf{1}$ . For centered data,  $T = 0$  and the determinant is  $mS > 0$ . Therefore  $\mathbf{g}_1$  and  $\mathbf{g}_3$  are linearly independent and  $\text{rank}(G) \geq 2$ .

**Conclusion.**  $G = \frac{1}{m} \begin{pmatrix} m & 1.5T & T \\ 1.5T & 2.25S & 1.5S \\ T & 1.5S & S \end{pmatrix}$  and  $\text{rank}(G) = 2$ . One dimension is lost because bedrooms and bathrooms carry the same information. The null direction is  $(0, 2, -3)^T$ , meaning “2×bedrooms – 3×bathrooms”, which is identically zero in the data.

### 3.2 Eigenvector for the largest eigenvalue of $G$ (5 points)

$G$  is symmetric positive semidefinite, so it has an orthonormal eigenbasis with nonnegative eigenvalues. One eigenpair is already known from 3.1:  $\lambda = 0$  with eigenvector  $\mathbf{n}/\|\mathbf{n}\| = (0, 2, -3)^T/\sqrt{13}$ . The other two eigenvectors must be orthogonal to  $\mathbf{n}$ . The space orthogonal to  $\mathbf{n}$  is spanned by the orthonormal pair

$$\mathbf{e}_1 = (1, 0, 0)^T \text{ (intercept)}, \quad \mathbf{u} = \frac{1}{\sqrt{13}}(0, 3, 2)^T \text{ (bedroom/bathroom ratio 1.5 : 1)}.$$

**Case  $T = 0$  (centered features, as stated in the problem).**  $G$  is block diagonal. The lower-right  $2 \times 2$  block is a rank-one outer product:

$$\frac{S}{m} \begin{pmatrix} 2.25 & 1.5 \\ 1.5 & 1 \end{pmatrix} = \frac{S}{m} \mathbf{b}\mathbf{b}^T, \quad \mathbf{b} = \begin{pmatrix} 1.5 \\ 1 \end{pmatrix}.$$

For any rank-one matrix  $\mathbf{b}\mathbf{b}^T$ ,  $(\mathbf{b}\mathbf{b}^T)\mathbf{b} = \mathbf{b}(\mathbf{b}^T\mathbf{b}) = \|\mathbf{b}\|^2\mathbf{b}$ , so  $\mathbf{b}$  is an eigenvector with eigenvalue  $\|\mathbf{b}\|^2 = 2.25 + 1 = 3.25$ . Every vector orthogonal to  $\mathbf{b}$  is mapped to  $\mathbf{0}$ . Hence

$$G\mathbf{u} = \frac{3.25S}{m}\mathbf{u}, \quad G\mathbf{e}_1 = 1 \cdot \mathbf{e}_1, \quad G\mathbf{n} = 0 \cdot \mathbf{n}.$$

We can check this directly:  $G(0, 3, 2)^T = (0, \frac{S}{m}(6.75 + 3), \frac{S}{m}(4.5 + 2))^T = \frac{3.25S}{m}(0, 3, 2)^T$ . ✓

The eigenvalues are  $\{3.25S/m, 1, 0\}$ . The feature direction  $\mathbf{u}$  has the largest eigenvalue whenever  $3.25S/m > 1$ , i.e.  $S/m > 1/3.25 \approx 0.31$ . This holds for standardized features, for which  $S/m = \widehat{\text{Var}}(x_2) = 1$  and  $\lambda_1 = 3.25$ . It also holds whenever the bathroom count has sample variance above about 0.31. Thus

$$\mathbf{v}_1 = \frac{1}{\sqrt{13}} \begin{pmatrix} 0 \\ 3 \\ 2 \end{pmatrix} \approx \begin{pmatrix} 0 \\ 0.832 \\ 0.555 \end{pmatrix}, \quad \lambda_1 = \frac{3.25S}{m} = 0.325S$$

The eigenvector is unique up to sign. If instead  $S/m < 1/3.25$ , the largest eigenvalue would be the constant intercept direction  $\mathbf{e}_1$ . That direction carries no information about how houses differ, so the relevant “feature-combining” eigenvector is still  $\mathbf{u}$ .

**General case  $T \neq 0$  (for completeness).** We restrict  $G$  to  $\text{span}\{\mathbf{e}_1, \mathbf{u}\}$ . Using  $(1.5, 1) = \frac{\sqrt{13}}{2} \cdot \frac{(3, 2)}{\sqrt{13}}$ ,

$$G\mathbf{e}_1 = \mathbf{e}_1 + c\mathbf{u}, \quad G\mathbf{u} = c\mathbf{e}_1 + \frac{3.25S}{m}\mathbf{u}, \quad c = \frac{\sqrt{13}}{2} \frac{T}{m},$$

so in the basis  $\{\mathbf{e}_1, \mathbf{u}\}$ ,  $G$  acts as  $M = \begin{pmatrix} 1 & c \\ c & 3.25S/m \end{pmatrix}$ . The largest root of  $\det(M - \lambda I) = 0$  is

$$\lambda_1 = \frac{\left(1 + \frac{3.25S}{m}\right) + \sqrt{\left(1 - \frac{3.25S}{m}\right)^2 + \frac{13T^2}{m^2}}}{2}, \quad \mathbf{v}_1 \propto c\mathbf{e}_1 + (\lambda_1 - 1)\mathbf{u}.$$

Setting  $T = 0$  gives  $c = 0$ , and this reduces to  $\mathbf{v}_1 = \mathbf{u}$  when  $3.25S/m > 1$ , matching the boxed result. In every case the feature part of  $\mathbf{v}_1$  points along  $(3, 2)$ , so the weight on bedrooms is always  $1.5\times$  the weight on bathrooms.

**Conclusion.** With centered features, the leading eigenvector is  $\mathbf{v}_1 = (0, 3, 2)^T/\sqrt{13}$ , with eigenvalue  $3.25S/m$ . It puts *zero* weight on the intercept and combines bedrooms and bathrooms with weights in ratio  $3 : 2 = 1.5 : 1$ .

### 3.3 Interpretation for house-price prediction (5 points)

- **The eigenvector defines a single “house size” feature.** Projecting a house onto  $\mathbf{v}_1$  gives the score

$$z = \frac{3x_1 + 2x_2}{\sqrt{13}} = \frac{3(1.5x_2) + 2x_2}{\sqrt{13}} = \frac{\sqrt{13}}{2} x_2 = \frac{\sqrt{13}}{3} x_1.$$

This is an overall size index that combines bedrooms and bathrooms in the same  $1.5 : 1$  ratio in which they occur in the data. Bedrooms get the larger weight simply because they vary  $1.5\times$  as much.

- **Why this direction captures the dominant variation.** Because  $x_1 = 1.5x_2$  for every house, all data points  $(x_{1,i}, x_{2,i})$  lie on a single line through the origin with direction  $(1.5, 1) \propto (3, 2)$ . The line passes through the origin because the features are centered. All the spread in the data is along this line, and its variance is  $\frac{1}{m} \sum_i z_i^2 = 3.25S/m$ , the eigenvalue  $\lambda_1$ . The perpendicular direction  $(2, -3)$  carries zero variance ( $\lambda = 0$ ). Since the two features are perfectly correlated, the data really have only *one* degree of freedom. PCA finds exactly that direction, and one principal component explains 100% of the feature variation.
- **Consequence for the regression.** The zero eigenvalue with eigenvector  $\mathbf{n} = (0, 2, -3)^T$  means  $XX^T$  is singular. The least-squares normal equations  $XX^T\boldsymbol{\beta} = X\mathbf{y}$  therefore have infinitely many solutions. For any  $t$ , the coefficient vector  $\boldsymbol{\beta} + t(0, 2, -3)^T$  gives identical predictions, because  $2x_{1,i} - 3x_{2,i} = 0$  for every house. The model cannot tell whether price is driven by bedrooms or bathrooms:  $\beta_1$  and  $\beta_2$  are not individually identifiable, and with nearly (rather than perfectly) correlated features they would have huge variance.
- **How PCA fixes it.** Regressing price on the single principal-component score,  $y = \beta_0 + \gamma z + \varepsilon$ , avoids the singularity entirely. The fitted  $\gamma$  is unique and has a clear meaning: the price change

per unit increase in overall house size. The implied coefficients  $\beta_1 = 3\gamma/\sqrt{13}$  and  $\beta_2 = 2\gamma/\sqrt{13}$  are then fixed in the natural 3 : 2 ratio. Dropping the zero-variance direction loses no information, since that direction is constant (zero) in the data.

- **The intercept.** The intercept direction  $\mathbf{e}_1$  is an eigenvector with eigenvalue 1 (when  $T = 0$ ). It only represents the constant baseline price level and is decoupled from the features once they are centered. This is why  $\mathbf{v}_1$  puts zero weight on it.

**In plain language:** bedrooms and bathrooms are redundant measurements of one underlying quantity, the size of the house. The leading eigenvector  $(0, 3, 2)/\sqrt{13}$  is the recipe for that size index. It captures all of the variation in the features, while the orthogonal combination “2×bedrooms − 3×bathrooms” has no variation and hence no predictive information. Using the principal component instead of both raw features gives a stable, interpretable regression.

#### 4. PCA: Food consumption in European countries (10 points)

**Implementation (own code, `pca.py`).** For a data matrix  $X \in \mathbb{R}^{m \times n}$  (one sample per row), `PCA.fit_transform`:

1. centers the data,  $X_c = X - \mathbf{1}\mu^T$  with  $\mu = \frac{1}{m} \sum_i \mathbf{x}_i$ ;
2. forms the sample covariance  $C = \frac{1}{m} X_c^T X_c \in \mathbb{R}^{n \times n}$ ;
3. computes the eigendecomposition of  $C$  with `numpy.linalg.eigh` ( $C$  is symmetric) and sorts eigenpairs by decreasing eigenvalue;
4. returns the projections  $Z = X_c W$ , where  $W = [\mathbf{w}_1, \mathbf{w}_2]$  holds the top two unit-norm eigenvectors. There is no further rescaling.

The class `FoodConsumptionPCA` (`food_consumption.py`) loads `food-consumption.csv` into a  $16 \times 20$  array and calls this routine on either the array or its transpose.

##### 4.1 Countries as samples, foods as features (5 points)

**Data matrix set-up.** Each *row* is a country and each *column* is a food item:

$$X \in \mathbb{R}^{16 \times 20}, \quad X_{ij} = \text{consumption of food } j \text{ in country } i.$$

Each country is a data point  $\mathbf{x}_i \in \mathbb{R}^{20}$  whose features are its 20 food consumptions. The covariance is  $20 \times 20$ , and the principal directions  $\mathbf{w}_1, \mathbf{w}_2 \in \mathbb{R}^{20}$  are weight vectors that combine the 20 food consumptions. The 2-D representation of country  $i$  is  $(\mathbf{w}_1^T(\mathbf{x}_i - \mu), \mathbf{w}_2^T(\mathbf{x}_i - \mu))$ . The first two PCs explain 33.0% and 19.3% of the variance (52.3% together).

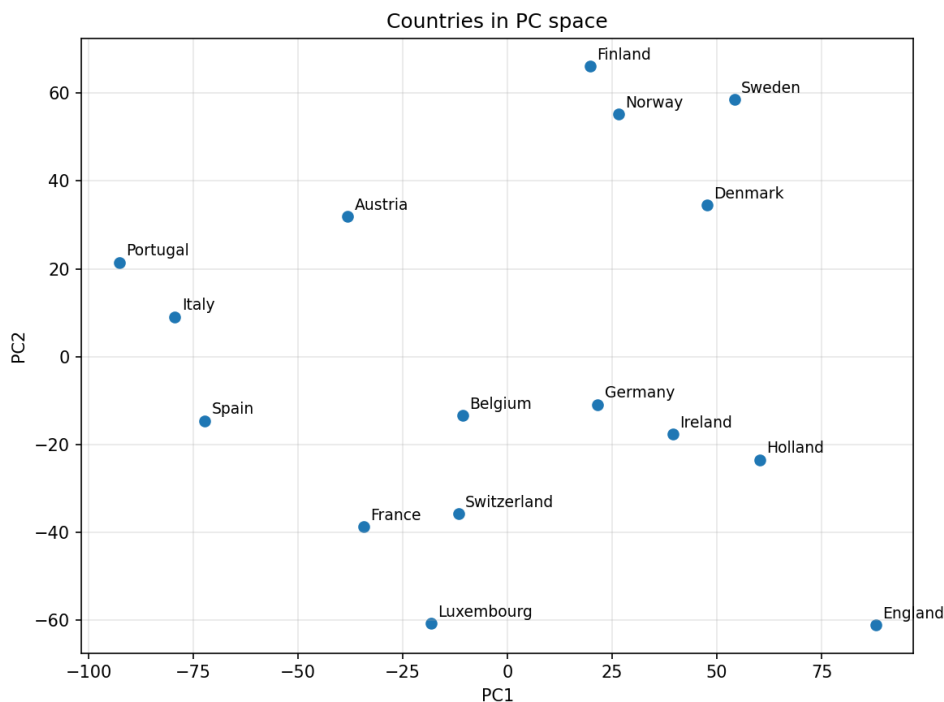


Figure 1: Countries projected onto the first two principal components (country-as-sample PCA).

**Largest loadings.** These explain the axes:

- **PC1:** most negative for garlic ( $-0.57$ ) and olive oil ( $-0.33$ ); most positive for jam ( $0.34$ ), tinned fruit ( $0.30$ ), tin soup ( $0.29$ ) and tea ( $0.26$ ).
- **PC2:** most positive for crisp bread ( $0.38$ ), real coffee ( $0.20$ ) and frozen fish ( $0.14$ ); most negative for instant coffee ( $-0.49$ ), tinned fruit ( $-0.34$ ), garlic ( $-0.30$ ) and powder soup ( $-0.26$ ).

**Patterns observed.**

- **PC1 separates Mediterranean from northern/Anglo diets.** Portugal, Italy and Spain sit far to the left (heavy garlic and olive oil). England, Holland, Ireland and Denmark sit to the right (jam, tinned fruit, tinned soup, tea).
- **PC2 isolates the Nordic countries.** Finland, Sweden, Norway and Denmark form a tight cluster at the top, driven by crisp bread, real coffee and frozen fish. England and Luxembourg are at the bottom (instant coffee, tinned fruit, powder soup).
- **Central Europe is in the middle.** Germany, Belgium, Switzerland and France lie near the origin, with near-average consumption on both axes.
- **Outliers and geography.** England is an outlier in the lower right (very high instant coffee, tinned fruit and tea; low garlic). Austria plots between the Mediterranean and Nordic groups. Overall the scatter roughly reproduces the geography of Europe (south, north, centre), so food culture is strongly regional.

## 4.2 Food items as samples, countries as features (5 points)

**Data matrix set-up.** We transpose the data:

$$X^T \in \mathbb{R}^{20 \times 16}, \quad (X^T)_{ji} = \text{consumption of food } j \text{ in country } i.$$

Each food item is a data point in  $\mathbb{R}^{16}$ , described by its consumption across the 16 countries. The covariance is  $16 \times 16$ , and the principal directions are weight vectors that combine country consumptions. The first two PCs explain 60.8% and 14.2% of the variance (75.0% together).

**Patterns observed.**

- **PC1 measures overall popularity across Europe.** Every loading of  $w_1$  is positive (from 0.15 for Sweden to 0.31 for Luxembourg), so PC1 is essentially a weighted average of consumption over all countries.
  - Right side: staples eaten everywhere, such as real coffee, tea, butter, margarine, apples, oranges and biscuits.
  - Left side: niche items with low consumption almost everywhere, such as potatoes, frozen vegetables, frozen fish, sweetener, tin soup, crisp bread and yoghurt.
- **PC2 contrasts southern and northern countries.**  $w_2$  loads positively on Portugal ( $0.38$ ), Spain ( $0.36$ ), Italy ( $0.33$ ) and Luxembourg. It loads negatively on England ( $-0.35$ ), Sweden, Ireland and Holland.
  - Garlic and olive oil stand far above the rest, the signature Mediterranean foods.

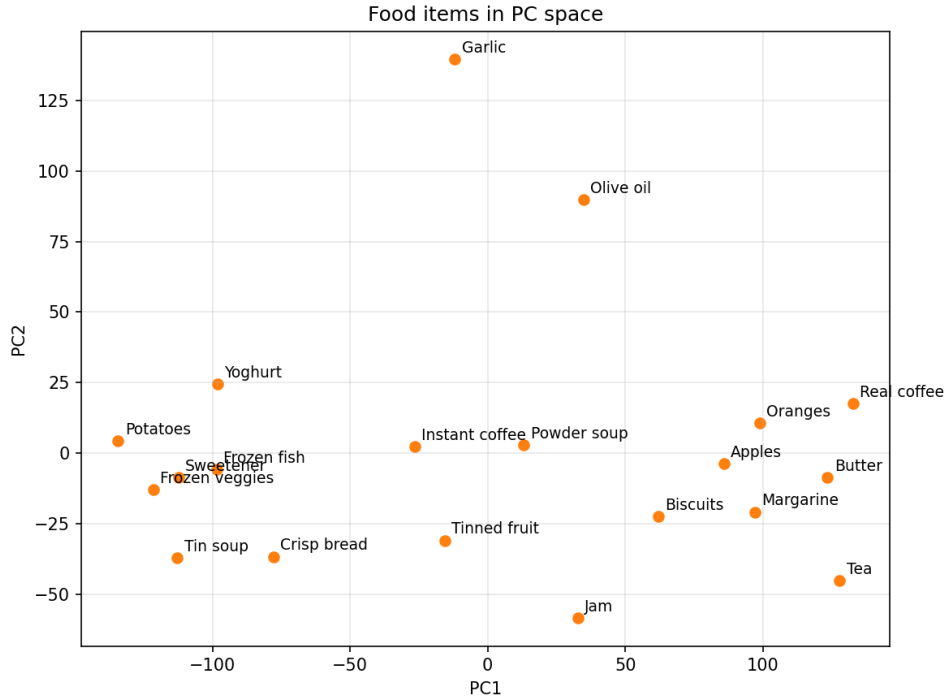


Figure 2: Food items projected onto the first two principal components (food-as-sample PCA).

- Jam, tea, tinned fruit, tin soup and crisp bread are at the bottom, typical of British, Irish and Nordic diets.
- **Similar points mean similar geographic patterns.** Nearby foods are consumed in a similar pattern across countries. For example, frozen fish, frozen vegetables and sweetener overlap, and margarine sits close to butter. This view complements 4.1: garlic and olive oil, which drove the Mediterranean countries apart there, are the outliers here too.

## 5. Order of faces using ISOMAP (20 points)

**Data and implementation (order\_of\_faces.py).** `isomap.mat` holds  $m = 698$  images of  $64 \times 64$  pixels, each a vector in  $\mathbb{R}^{4096}$ . The file stores images as columns, so the code transposes the matrix to put one image per row, and reshapes each vector to  $64 \times 64$  in column-major order for display. All pairwise Euclidean distances  $d_{ij} = \|\mathbf{x}_i - \mathbf{x}_j\|_2$  are computed once.

### 5.1 Nearest-neighbour graph (5 points)

**$\epsilon$ -graph.** Nodes  $i$  and  $j$  are joined by an edge of weight  $d_{ij}$  if and only if  $d_{ij} \leq \epsilon$ ; otherwise  $A_{ij} = 0$  (`get_adjacency_matrix`). Unlike  $K$ -ISOMAP, node degrees are not fixed.

**Choice of  $\epsilon$  (`get_best_epsilon`).** We pick the *smallest*  $\epsilon$  that makes the graph connected. This equals the longest edge of the minimum spanning tree of the complete distance graph: every MST edge is then present, and no smaller  $\epsilon$  can connect the endpoints of that longest edge. The code

confirms that `connected_components` returns one component. This gives

$$\epsilon^* = 10.41, \quad 7,606 \text{ edges}, \quad \text{average degree } 21.8 \text{ (min } 1, \text{ max } 77).$$

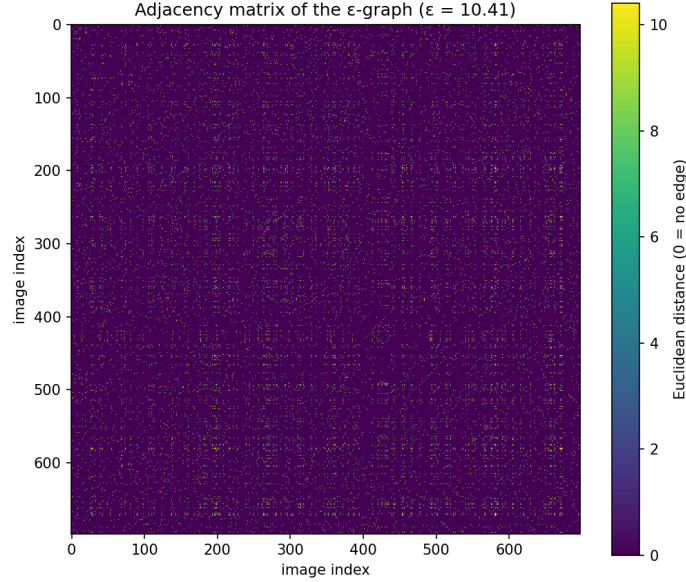


Figure 3: Weighted adjacency matrix of the  $\epsilon$ -graph ( $\epsilon = 10.41$ ). Coloured entries are edges, coloured by Euclidean distance; dark entries mean no edge. The matrix is sparse (about 3% of pairs) and symmetric.

**Observations.** In the graph drawing, neighbouring nodes have similar poses.

- The left-hand tail (e.g. #286) holds strongly turned, brightly lit faces.
- The top region (#474, #482, #593) holds faces tilted downward.
- The right and bottom regions (#506, #331, #437) hold the opposite head turns.

The graph is a connected “sheet” that follows the pose manifold. The dense core contains darker images (e.g. #39), which are close to each other in pixel space.

## 5.2 ISOMAP embedding (10 points)

**Algorithm (our own implementation, `isomap`).**

1. Build the weighted  $\epsilon$ -graph  $A$  (5.1).
2. Compute geodesic distances  $D_{ij}$  as all-pairs shortest-path lengths on  $A$ , using Dijkstra’s algorithm from SciPy’s `shortest_path`. An error is raised if any distance is infinite (disconnected graph).
3. Apply classical MDS with centering matrix  $H = I - \frac{1}{m}\mathbf{1}\mathbf{1}^T$ , and form the Gram matrix  $C = -\frac{1}{2}H(D \circ D)H$ , symmetrized numerically.
4. Take the two leading eigenpairs  $(\lambda_k, \mathbf{u}_k)$  of  $C$  and return  $Z = [\sqrt{\lambda_1}\mathbf{u}_1, \sqrt{\lambda_2}\mathbf{u}_2] \in \mathbb{R}^{698 \times 2}$ .

**Visual similarity and arrangement.** The arrangement closely matches Figure 1A of Tenenbaum et al. (2000).

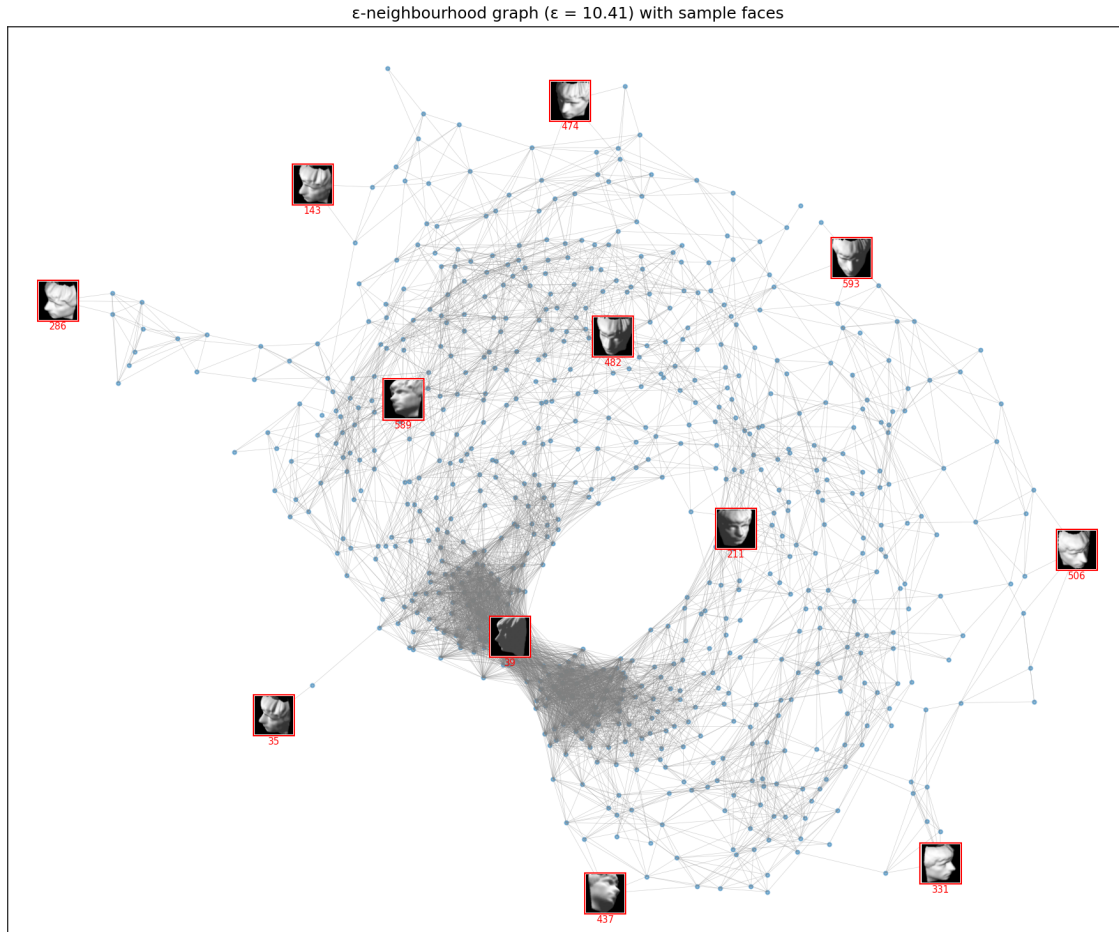


Figure 4: The  $\epsilon$ -graph drawn with a force-directed (Kamada–Kawai) layout, similar to the Gephi drawing in the lecture. Twelve nodes spread across the layout are shown with their face images (red boxes, with image index).

- **Component 1 is left–right pose.** Faces on the far left (e.g. #286, #474, #254) are turned toward one side. Faces near  $x \approx 0$  (#285, #681) are roughly frontal. Faces on the far right (#506, #180, #6) are turned toward the other side. The first row of the quantile strip shows the turn progressing monotonically. The correlation of component 1 with the horizontal intensity centroid of the image is  $-0.64$ .
- **Component 2 is up–down pose and lighting.** The upper part holds brighter faces tilted downward (#305, #285, #506). The lower part holds darker faces and faces tilted upward (#69, #159, #6). Component 2 correlates  $0.69$  with mean image brightness; the paper’s third ISOMAP axis is lighting direction, which is here partly folded into the 2-D picture.
- **Local continuity.** Images close in the embedding look alike, so walking through the scatter traces a continuous change of head orientation, as in the paper.

#### Tuning the bandwidth $\epsilon$ .

- *Lower bound (connectivity).* If  $\epsilon$  is too small the graph splits into components, some geodesic

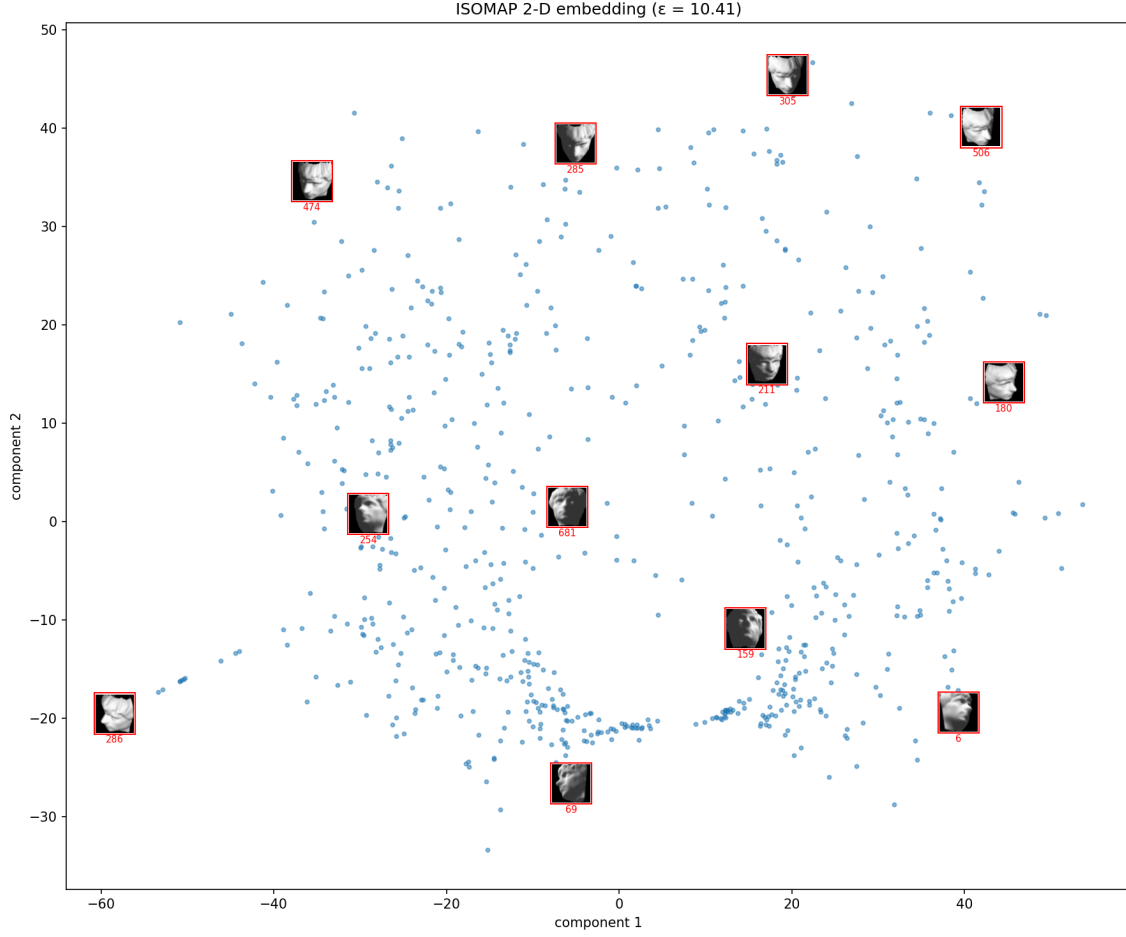


Figure 5: Two-dimensional ISOMAP embedding ( $\epsilon = 10.41$ ). Twelve images spread over the embedding (farthest-point sampling) are shown at their locations.

distances become  $\infty$ , and MDS fails. The smallest feasible value is the longest MST edge,  $\epsilon = 10.41$ , which our method selects.

- *Upper bound (no short-circuits).* If  $\epsilon$  is too large, edges jump across the manifold. Geodesic distances then collapse to straight-line Euclidean distances, and ISOMAP degenerates toward PCA/MDS.
- *Procedure.* Start from the connectivity threshold and increase  $\epsilon$  gradually. Track the average degree, and check that faces in the embedding vary smoothly in pose, using the colour-coded comparison below. Keep the smallest  $\epsilon$  that is connected and gives a smooth, unfolded layout.

**What the comparison shows.** For  $\epsilon = 10.41$  and 12 (average degree 22–40) the colour gradient stays a smooth left-to-right sweep, so the pose ordering is stable. At  $\epsilon = 15$  (average degree 105) the ordering survives but the shape distorts. At  $\epsilon = 20$  (average degree 327) the colours are scrambled and red and blue points overlap, the signature of short-circuit edges. This supports choosing  $\epsilon$  near the connectivity threshold, roughly 10–12.

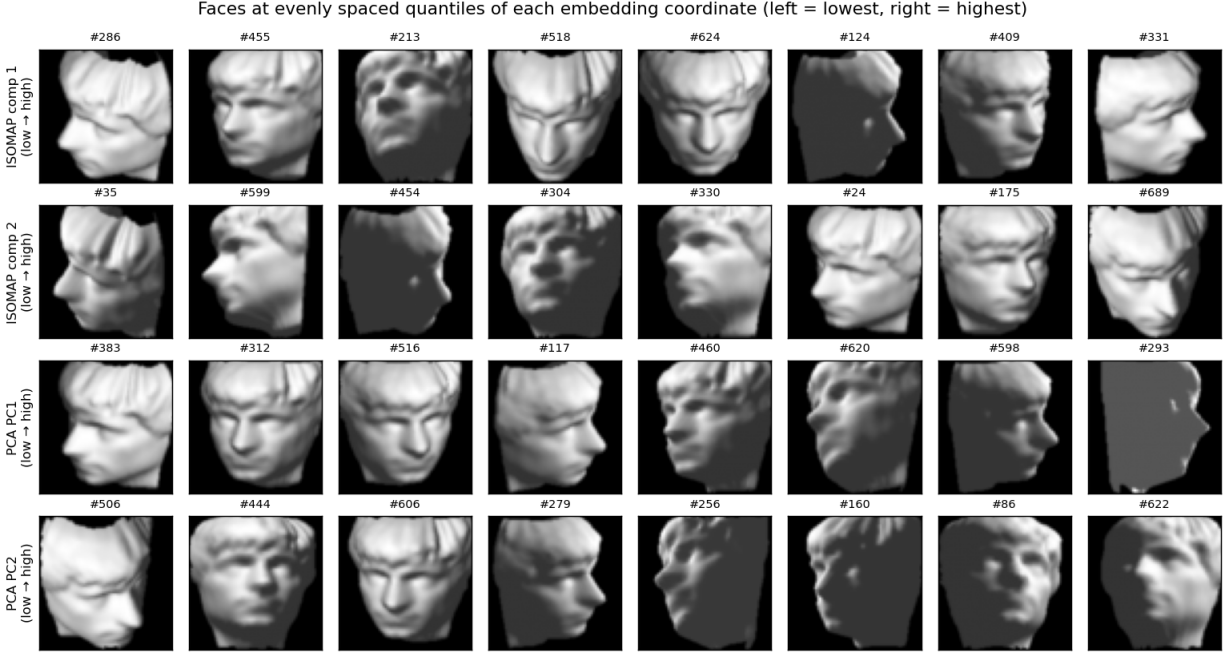


Figure 6: Faces at eight evenly spaced quantiles of each coordinate (rows 1–2: ISOMAP; rows 3–4: PCA from 5.3). Moving left to right in row 1 shows a smooth left-to-right head turn.

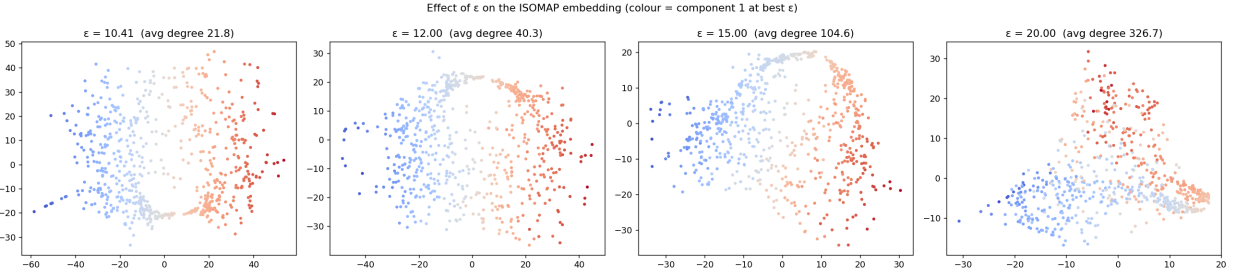


Figure 7: ISOMAP embeddings for  $\epsilon \in \{10.41, 12, 15, 20\}$ . Points are coloured by their component-1 value at  $\epsilon = 10.41$ .

### 5.3 PCA projection and comparison (5 points)

PCA (`OrderOfFaces.pca`) centers the  $698 \times 4096$  data and takes the top-2 right singular vectors of the centered matrix. These equal the top eigenvectors of the sample covariance, the same subspace as our Q4 PCA. The data are projected onto them. The first two PCs explain 27.1% and 21.6% of the pixel variance.

**Is ISOMAP more meaningful than PCA? Yes.**

- **PC1 mostly measures brightness, not pose.** PC1 correlates  $-0.95$  with mean image brightness. Bright faces (#383, #511, #518) sit on the left and dark faces (#293, #399, #651) on the right, whatever their pose (rows 3–4 of the quantile strip). PC2 is driven by where the bright

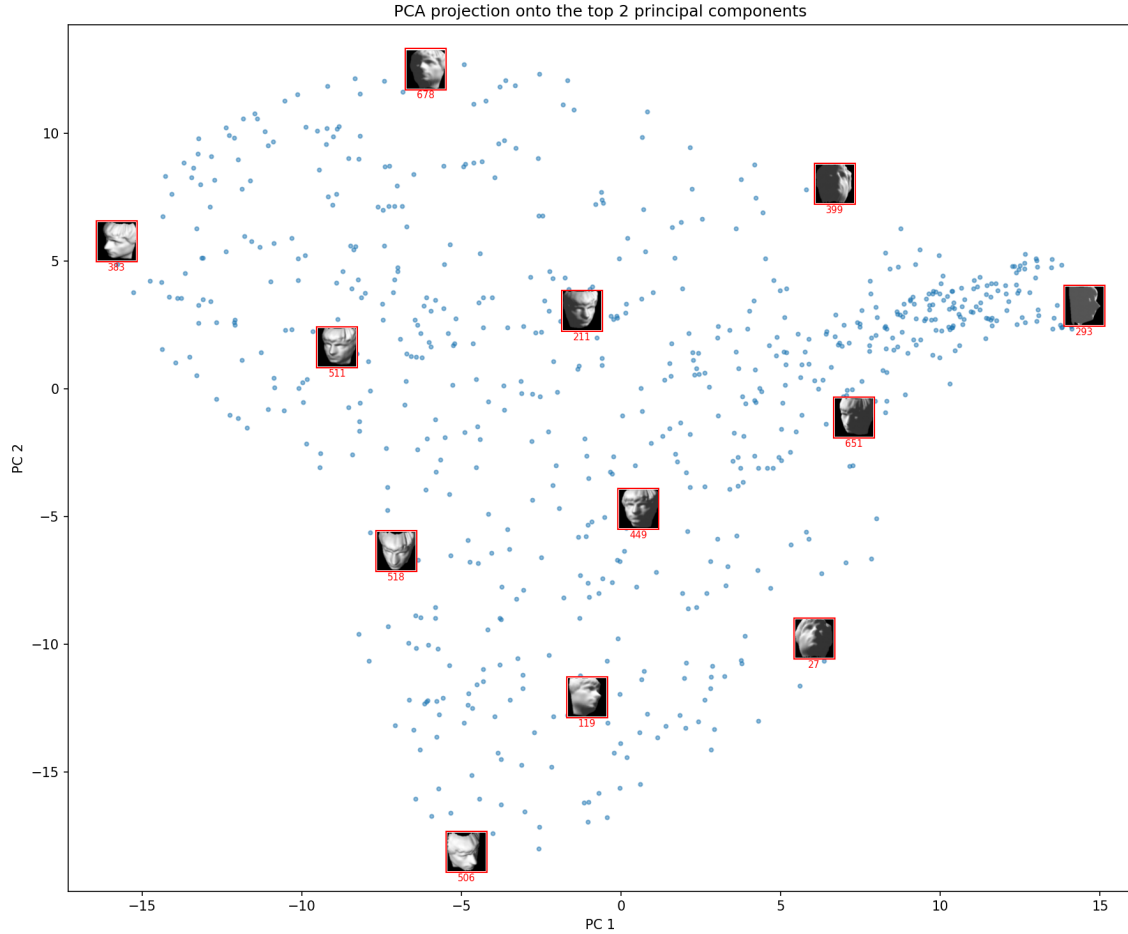


Figure 8: Projection of the face images onto the top two principal components, with twelve highlighted images.

pixels lie horizontally, which mixes head turn with lighting side.

- **Very different poses end up next to each other.** Dimly lit faces with very different poses, such as the near-frontal #651, the turned #399 and the profile #293, are grouped together on the right because they share low brightness. Turned faces appear on both sides of the plot. There is no single coordinate along which the head turns smoothly.
- **Why the methods differ.** The images lie on a curved, low-dimensional manifold (pose and lighting) inside  $\mathbb{R}^{4096}$ . PCA only finds the linear subspace of maximum pixel variance, and much of that variance comes from global intensity changes. ISOMAP preserves geodesic distances measured *along* the manifold through the  $\epsilon$ -graph. It therefore “unrolls” the manifold and recovers the intrinsic pose coordinates, which gives a more interpretable and meaningful ordering of the faces.

## 6. Eigenfaces and simple face recognition (20 points)

**Pre-processing (eigen\_faces.py).** Each Yale face GIF ( $320 \times 243$  pixels) is converted to grayscale and downsampled by a factor of 4 with bilinear interpolation (`PIL.Image.resize`) to  $80 \times 60$  (width $\times$ height). It is then vectorized to  $\mathbb{R}^{4800}$ .

**Training sets.** The test images `subject01-test.gif` and `subject02-test.gif` are excluded.

- Subject 1:  $m_1 = 10$  images (glasses, happy, leftlight, noglasses, normal, rightlight, sad, sleepy, surprised, wink).
- Subject 2:  $m_2 = 9$  images (the same conditions without “surprised”).

Each data matrix has one vectorized picture per row.

**PCA.** For each subject we compute the mean face  $\mu_i$  and the covariance  $C_i = \frac{1}{m_i} X_{c,i}^T X_{c,i} \in \mathbb{R}^{4800 \times 4800}$ . The eigenfaces are the top unit-norm eigenvectors of  $C_i$ , reshaped to  $60 \times 80$  for display.

### 6.1 Top-6 eigenfaces (10 points)

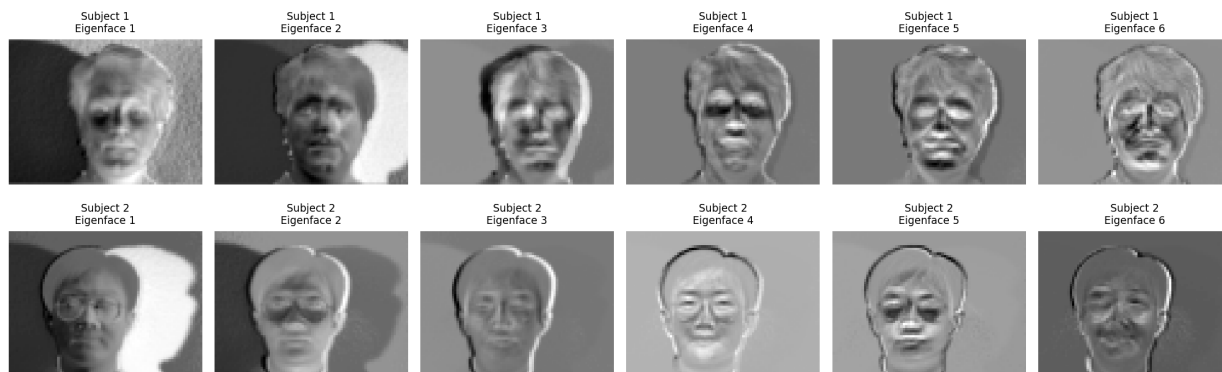


Figure 9: First six eigenfaces of Subject 1 (top row) and Subject 2 (bottom row).

| Variance explained (%) | EF1  | EF2  | EF3 | EF4 | EF5 | EF6 | Top 6 total |
|------------------------|------|------|-----|-----|-----|-----|-------------|
| Subject 1              | 51.5 | 31.5 | 9.2 | 3.2 | 2.1 | 1.0 | 98.5        |
| Subject 2              | 54.2 | 31.7 | 9.1 | 2.0 | 1.4 | 0.9 | 99.3        |

**Patterns in the eigenfaces.**

- **Eigenfaces 1–2 capture illumination.** They show strong left/right light–dark contrasts on the face and the background shadow, with the background bright on one side and dark on the other. These directions are driven by the *leftlight* and *rightlight* images and together explain  $\approx 83$ – $86\%$  of the variance. Illumination is by far the largest source of variation within a person.
- **Eigenfaces 3–6 capture local features and expressions.** They concentrate on the eyes, eyebrows, nose, mouth and face contour, showing bright/dark patterns around the eyes and

mouth. These correspond to expression changes (happy, sad, surprised, wink, sleepy) and to glasses vs. no glasses. For Subject 2 the glasses frame is clearly visible in eigenfaces 1, 2, 4 and 5.

- **Identity is still visible.** Every eigenface keeps that person’s head and hair outline, so each subject’s eigenfaces form a subject-specific basis. The variance spectrum decays quickly: six eigenfaces explain  $\geq 98.5\%$  of the training variance.

## 6.2 Face recognition with projection residuals (5 points)

For eigenface subject  $i$  with mean face  $\boldsymbol{\mu}_i$  and single top eigenface  $\mathbf{e}_i$  ( $\|\mathbf{e}_i\| = 1$ ), and test image  $\mathbf{t}_j$ , the score is

$$s_{ij} = \|\mathbf{c}_{ij} - \mathbf{e}_i \mathbf{e}_i^T \mathbf{c}_{ij}\|_2^2, \quad \mathbf{c}_{ij} = \mathbf{t}_j - \boldsymbol{\mu}_i.$$

This is the squared distance from the test image to the 1-D affine “face space”  $\{\boldsymbol{\mu}_i + a \mathbf{e}_i\}$  of subject  $i$ .

|                                 | Test 1 (subject01-test)          | Test 2 (subject02-test)          |
|---------------------------------|----------------------------------|----------------------------------|
| Subject 1 eigenface ( $i = 1$ ) | $s_{11} = \mathbf{6,301,725.88}$ | $s_{12} = 38,709,139.90$         |
| Subject 2 eigenface ( $i = 2$ ) | $s_{21} = 31,800,926.12$         | $s_{22} = \mathbf{3,366,171.29}$ |

**Recognition rule.** Assign test image  $j$  to the subject whose face space reconstructs it best,  $\hat{i}(j) = \arg \min_i s_{ij}$ .

- Test image 1:  $s_{11} = 6.30 \times 10^6 < s_{21} = 3.18 \times 10^7$ , so it is classified as **Subject 1** (correct).
- Test image 2:  $s_{22} = 3.37 \times 10^6 < s_{12} = 3.87 \times 10^7$ , so it is classified as **Subject 2** (correct).

A small residual means the test image is well described by that subject’s mean face plus eigenface, i.e. it “looks like” that person. Both test images are recognized correctly, with residual ratios of about  $5\times$  and  $11.5\times$  between the wrong and the right subject.

## 6.3 Does it work well, and how to improve it (5 points)

**Assessment.** On this task the method works: both test faces are classified correctly with a wide margin. The test is weak, however.

- Only two subjects and two test images are used, so the accuracy estimate has essentially no statistical power.
- Most of the discrimination comes from the *mean face*, not the eigenface. The plain squared distances to the mean faces are  $\|\mathbf{t}_1 - \boldsymbol{\mu}_1\|^2 = 6.43 \times 10^6$  vs.  $\|\mathbf{t}_1 - \boldsymbol{\mu}_2\|^2 = 4.17 \times 10^7$ , and  $4.22 \times 10^6$  vs.  $4.67 \times 10^7$  for test image 2. Projecting onto one eigenface lowers  $s_{11}$  by only 2%. The top eigenface mainly models *lighting*, which helps little for a normally lit test image.
- Because the first eigenfaces encode illumination, the method can be fooled by lighting. A strongly side-lit image of one person could be closer to another person’s lighting eigenface than to its own face space.
- Raw pixel PCA is also sensitive to misalignment, scale and background.

### Possible improvements.

1. **Use more eigenfaces.** Use the top  $k$  eigenfaces (e.g.  $k = 3-6$ , chosen by explained variance or cross-validation) and compute  $\|\mathbf{c} - E_k E_k^T \mathbf{c}\|^2$ .
2. **Normalize illumination.** Apply histogram equalization or per-image mean/variance normalization, or drop the first 1–3 lighting eigenfaces, a known trick from Belhumeur et al.
3. **Build one shared face space.** Fit PCA on all subjects together and classify the projections with nearest neighbour or a classifier. Better still, use Fisherfaces (LDA), which explicitly maximizes between-subject relative to within-subject variance.
4. **Align and crop faces.** Register faces using eye locations and crop out the background, so pixels correspond across images.
5. **Evaluate on more data.** Test on more subjects and images, using leave-one-out cross-validation, and add a rejection threshold on  $\min_i s_{ij}$  to detect unknown faces.
6. **Use better features.** Use local features (LBP, HOG) or learned CNN embeddings, which are far more robust to pose and illumination.

## 7. To subtract or not to subtract (Bonus: 10 points)

**Answer: in general  $\tilde{\mathbf{w}}_1 \neq \mathbf{w}_1$ .** They coincide only in special cases (e.g.  $\boldsymbol{\mu} = \mathbf{0}$ , or  $\boldsymbol{\mu}$  itself is an eigenvector of  $C$  along  $\mathbf{w}_1$ , as made precise below).

**Step 1: Relating  $\tilde{C}$  to  $C$ .** Expanding  $C$ ,

$$C = \frac{1}{m} \sum_{i=1}^m (\mathbf{x}^i - \boldsymbol{\mu})(\mathbf{x}^i - \boldsymbol{\mu})^T = \frac{1}{m} \sum_i \mathbf{x}^i \mathbf{x}^{iT} - \boldsymbol{\mu} \left( \frac{1}{m} \sum_i \mathbf{x}^i \right)^T - \left( \frac{1}{m} \sum_i \mathbf{x}^i \right) \boldsymbol{\mu}^T + \boldsymbol{\mu} \boldsymbol{\mu}^T = \tilde{C} - \boldsymbol{\mu} \boldsymbol{\mu}^T,$$

using  $\frac{1}{m} \sum_i \mathbf{x}^i = \boldsymbol{\mu}$ . Hence

$$\boxed{\tilde{C} = C + \boldsymbol{\mu} \boldsymbol{\mu}^T.}$$

$\tilde{C}$  is the covariance plus a rank-one term pointing along the mean.

**Step 2: When are the leading eigenvectors the same?** Suppose  $\mathbf{w}_1$  (unit,  $C\mathbf{w}_1 = \lambda_1 \mathbf{w}_1$ ) were also an eigenvector of  $\tilde{C}$ . Then

$$\tilde{C}\mathbf{w}_1 = \lambda_1 \mathbf{w}_1 + \boldsymbol{\mu}(\boldsymbol{\mu}^T \mathbf{w}_1).$$

This is a multiple of  $\mathbf{w}_1$  iff  $\boldsymbol{\mu}(\boldsymbol{\mu}^T \mathbf{w}_1) \parallel \mathbf{w}_1$ , i.e. iff either  $\boldsymbol{\mu}^T \mathbf{w}_1 = 0$  ( $\boldsymbol{\mu} \perp \mathbf{w}_1$ ) or  $\boldsymbol{\mu} \parallel \mathbf{w}_1$ . So for a generic mean (neither parallel nor orthogonal to  $\mathbf{w}_1$ )  $\mathbf{w}_1$  is *not even an eigenvector* of  $\tilde{C}$ , so  $\tilde{\mathbf{w}}_1 \neq \pm \mathbf{w}_1$ .

Even in the two special cases the top eigenvector can change:

- $\boldsymbol{\mu} \parallel \mathbf{w}_1$ :  $\tilde{C}\mathbf{w}_1 = (\lambda_1 + \|\boldsymbol{\mu}\|^2)\mathbf{w}_1$ , and every other eigenvalue is unchanged, so  $\tilde{\mathbf{w}}_1 = \mathbf{w}_1$ .
- $\boldsymbol{\mu} \perp \mathbf{w}_1$ :  $\mathbf{w}_1$  keeps eigenvalue  $\lambda_1$ , but  $\tilde{C}$  has Rayleigh quotient  $\frac{\boldsymbol{\mu}^T \tilde{C} \boldsymbol{\mu}}{\|\boldsymbol{\mu}\|^2} \geq \|\boldsymbol{\mu}\|^2$  along  $\boldsymbol{\mu}$ . When  $\|\boldsymbol{\mu}\|^2 > \lambda_1$ ,  $\tilde{\lambda}_1 > \lambda_1$  and  $\tilde{\mathbf{w}}_1$  points (roughly) along  $\boldsymbol{\mu}$ , not  $\mathbf{w}_1$ .

**Step 3: A concrete counterexample.** Take  $m = 2$  points in  $\mathbb{R}^2$ :  $\mathbf{x}^1 = (9, 1)^T$ ,  $\mathbf{x}^2 = (11, -1)^T$ . Then  $\boldsymbol{\mu} = (10, 0)^T$ ,  $\mathbf{x}^1 - \boldsymbol{\mu} = (-1, 1)^T$ ,  $\mathbf{x}^2 - \boldsymbol{\mu} = (1, -1)^T$ , and

$$C = \frac{1}{2} \left[ \begin{pmatrix} 1 & -1 \\ -1 & 1 \end{pmatrix} + \begin{pmatrix} 1 & -1 \\ -1 & 1 \end{pmatrix} \right] = \begin{pmatrix} 1 & -1 \\ -1 & 1 \end{pmatrix}, \quad \mathbf{w}_1 = \frac{1}{\sqrt{2}}(1, -1)^T, \quad \lambda_1 = 2.$$

Without centering,

$$\tilde{C} = C + \boldsymbol{\mu} \boldsymbol{\mu}^T = \begin{pmatrix} 101 & -1 \\ -1 & 1 \end{pmatrix}, \quad \tilde{\lambda}_1 = 51 + \sqrt{2501} \approx 101.01, \quad \tilde{\mathbf{w}}_1 \approx (0.99995, -0.0100)^T.$$

$\mathbf{w}_1$  is the direction of the data's spread (the  $45^\circ$  line), while  $\tilde{\mathbf{w}}_1$  is essentially  $(1, 0)^T$ , the direction of the mean. Indeed  $\tilde{C}\mathbf{w}_1 = \frac{1}{\sqrt{2}}(102, -2)^T \not\parallel \mathbf{w}_1$ , so  $\tilde{\mathbf{w}}_1 \neq \mathbf{w}_1$ .

**Conclusion.**  $\tilde{C} = C + \boldsymbol{\mu} \boldsymbol{\mu}^T$ , so  $\tilde{\mathbf{w}}_1 \neq \mathbf{w}_1$  in general. Without centering, the leading direction is pulled toward the mean  $\boldsymbol{\mu}$  (it maximizes the *second moment*  $\frac{1}{m} \sum (\mathbf{w}^T \mathbf{x}^i)^2 = \mathbf{w}^T C \mathbf{w} + (\mathbf{w}^T \boldsymbol{\mu})^2$ , not the variance). The two agree only when  $\boldsymbol{\mu} = \mathbf{0}$  or when  $\boldsymbol{\mu}$  happens to lie along  $\mathbf{w}_1$ . Mean subtraction is therefore needed for  $\mathbf{w}_1$  to be the direction of maximum variance.